KASPARRO

AI Representation Optimizer for Shopify

Product Document

Prepared by Aswin Kumar B S and Naveen D

One-line summary

Kasparro tells Shopify merchants exactly why AI shopping assistants are ignoring their products — and fixes it, automatically, in one click.

Product category	Shopify SaaS — AI content intelligence and optimisation
Core capability	Score, diagnose, fix, and track AI discoverability for every product in a Shopify catalog
Primary user	Shopify merchants ($100K–$5M ARR) with no dedicated SEO team
Secondary user	Shopify agencies and consultants managing multiple stores
Total features shipped	10 distinct AI-powered analysis and optimisation modules
AI providers integrated	Google Gemini (primary), OpenRouter, Groq, AWS Bedrock Nova, OpenAI GPT-4o-mini, Tavily, SerpAPI
Deployment	Vercel (frontend SPA) + Railway (persistent API) + Neon (PostgreSQL)
Hackathon	Kasparro Agentic Commerce Hackathon — April 2026

1. The Problem We Are Solving

The Structural Shift in Product Discovery

Consumer product discovery is migrating from keyword search to AI-generated answers. ChatGPT Shopping, Google AI Overviews, Perplexity, and Bing Copilot now answer product questions directly — no click required, no result page to rank on. These agents do not crawl and rank; they synthesise. A product whose content cannot be parsed, understood, and confidently cited by an AI agent does not appear in that buying journey at all.

This is not a future problem. It is happening now in product verticals like electronics, apparel, home goods, and supplements — categories where AI shopping assistants are already the dominant discovery surface for high-consideration purchases.

Why Existing Tools Fail Shopify Merchants

Traditional SEO tools — Ahrefs, SEMrush, Screaming Frog — measure backlinks, keyword density, and page speed. None of these signals matter to an AI agent deciding whether to cite a product. What matters for AI retrieval is fundamentally different: structured specifications, unambiguous benefit statements, schema markup, topical authority, FAQ coverage, and consistent brand voice. These are content quality signals, not technical SEO signals.

Shopify's native analytics show traffic and conversion. App Store tools audit for SEO basics (meta descriptions, alt text). No tool in the Shopify ecosystem today answers the question a merchant actually needs answered:

> **The unanswered question every Shopify merchant has**
>
> "ChatGPT is recommending products in my category to buyers every day. Why is it not recommending mine — and what exactly do I need to change?"

The Specific Gaps We Fill

- No diagnostic for AI invisibility — merchants cannot tell if their product content is AI-parseable
- No benchmark — merchants have no reference point for what a "good" AI-ready product description looks like in their category

- No action path — even when a merchant suspects the problem, there is no tooling to generate a better version and deploy it automatically
- No measurement — there is no way to verify whether an optimisation actually improved AI citation rates across real AI engines

Before and After Kasparro

Without Kasparro	With Kasparro
No idea why AI ignores products	5-dimension scored report with exact rule violations and ranked fixes
Must manually test each AI engine	5-platform GEO scan in one click — Tavily, Google, Bedrock, OpenAI, Gemini
Rewrites are guesswork	AI-generated rewrites grounded in detected violations, preserving brand voice
Cannot verify fixes worked	Shopify write verified by read-back; score updates only on confirmed sync
No competitive visibility	Competitor AI readiness scores across all 5 dimensions vs. your own
Must monitor trends manually	Automated weekly/monthly analysis with score-regression email alerts

2. Who We Are Building For

Primary Persona: The Mid-Market Shopify Merchant

Profile: A Shopify merchant generating $100K–$5M in annual revenue. They manage the store personally or with a team of 2–5. They know their products deeply but have no dedicated SEO, content, or data team. They are already AI-adjacent — they use ChatGPT to draft copy, Gemini to brainstorm campaigns — but they have no way to measure whether the AI-assisted content they are publishing is actually working.

Their daily reality: They add products, run ads, watch Google Analytics. When conversions drop, they suspect it might be the content but have no evidence. They do not know that their competitors restructured their product descriptions 6 months ago and are now appearing in ChatGPT Shopping recommendations while their own products are not.

Specific pains Kasparro eliminates for this persona:

- Hours spent manually reviewing each product page for quality issues
- Uncertainty about whether AI-written descriptions are structured correctly for AI retrieval
- No way to prioritise which product issues to fix first for maximum impact
- Having to copy-paste content from their store into ChatGPT manually to test AI responses
- No alert system when a competitor improves their AI score faster

Secondary Persona: The Shopify Agency or Consultant

Profile: A digital agency or freelance consultant managing 5–50 Shopify stores for clients. They need a repeatable audit process they can run across a portfolio, a clear report format they can present to clients, and an ROI narrative that justifies the engagement cost.

What this persona needs from Kasparro:

- A structured, branded-quality report they can present to non-technical clients
- Per-product scoring so they can prioritise which products to optimise first in a client engagement
- Score history with trend lines so they can show before/after improvement from their work
- CSV export for building client reporting dashboards in Google Sheets

Who We Are Not Building For (Yet)

Enterprise merchants with dedicated content operations teams and custom PIM systems are out of scope — they have the resources to build custom solutions. D2C brands on platforms other than Shopify (BigCommerce, WooCommerce) are a logical expansion but not in this build. B2B merchants selling to procurement teams rather than individual consumers have fundamentally different AI discovery patterns and would require a different scoring model.

3. The Complete Product — All Features Built

Kasparro ships with 10 distinct AI-powered analysis and optimisation modules. The table below lists every feature and its status. Each module is described in detail in the sub-sections that follow.

Feature Module	Status	What It Delivers
AI Readiness Score Engine	**Built**	0–100 score across 5 weighted dimensions for every product; store-level aggregate; 60/40 rule + AI blend
Intelligent Fix Engine	**Built**	AI-generated fixes for all high-severity gaps; single-click and bulk apply; editable before commit; Shopify write verification

Feature Module	Status	What It Delivers
GEO Scanner	**Built**	Real 5-platform citation scan (Tavily, SerpAPI, AWS Bedrock, OpenAI, Gemini); citation rate per engine; trend history
Competitor Intelligence	**Built**	Analyze up to 5 competitor Shopify domains; 5-dimension score comparison; actionable gap insights
LLMs.txt Generator	**Built**	Machine-readable store identity file for AI crawlers; one-click deploy to Shopify store pages
FAQ Schema Generator	**Built**	JSON-LD FAQ schema grounded in real policy text; ready to paste into Shopify theme
Product AI Q&A Test	**Built**	Per-product answerability score — tests whether product content answers 10 real buyer questions
AI Query Simulation	**Built**	Simulates AI shopping assistant queries against catalog; wouldRecommend rate; 24h cache with hash invalidation
Topical Authority Analyzer	**Built**	Clusters products into topic groups; coverage score per cluster; identifies content gaps in catalog
AI Perception Report	**Built**	Full narrative of how an AI agent perceives the store's brand; ambiguities, unanswered questions, strengths
Tag Optimizer	**Built**	Identifies products with weak or missing tags; suggests AI-taxonomy-aligned replacements
Listing Readiness Score	**Built**	A–F grade on 12 data-completeness checks; checks description, image, price, vendor, tags, policies
Scheduled Analysis	**Built**	Automated daily/weekly/monthly analysis; timezone support; pause/resume; idempotent retry
Email Reports	**Built**	Score snapshot emails on schedule; send only on change / on regression; configurable min-delta threshold
Score History & CSV Export	**Built**	Append-only score snapshots with area chart; paginated history; Google Sheets-compatible CSV download
Internal Link Recommendations	**Built**	AI-suggested cross-links between semantically related products in the catalog
Activity Feed	**Built**	Chronological log of all fix applications, analysis runs, and sync events with metadata

3.1 AI Readiness Score Engine

The score engine is the analytical foundation of the entire product. Every product in a merchant's catalog receives a numeric score from 0 to 100 across five independently-weighted dimensions. The store-level score is the catalog-weighted average of all product scores. Scores are stored as snapshots after each analysis, enabling trend tracking over time.

Dimension	What It Measures	Weight
Clarity	Title word count, sentence structure, NLP parse-friendliness, passive-voice detection, filler-phrase detection	25%
Completeness	Spec coverage, structured body content, comparison-readiness, dimension data, variant detail, feature density	25%
Trust	JSON-LD schema markup, review signal alignment, policy page presence (refund/shipping/privacy), vendor identity	20%
Tags	Tag count (minimum 5 required), format validity, AI-taxonomy alignment, comparison-query parsability	15%
Consistency	Cross-product brand voice coherence, pricing signal integrity, policy statement consistency	15%

Scoring blend: Each dimension's final score is a weighted blend — 60% from the deterministic rule engine (fast, auditable, explainable) and 40% from Google Gemini's quality assessment (nuanced, context-aware). This split was chosen because pure rule scores miss subtlety (a technically valid description can still be weak), while pure AI scores are non-deterministic across runs.

Rule engine: Approximately 40 deterministic rules fire against each product in under 5ms. Every rule produces: (a) a severity level — high, medium, or low; (b) a human-readable violation string with specific evidence ('Title has 2 words — minimum 6 required'); (c) an impact score that drives the fix priority ranking. Rules are grouped by dimension so every gap card maps directly to the dimension it affects.

3.2 Intelligent Fix Engine

Every high-severity gap detected by the score engine has a corresponding AI-generated fix pre-computed during the analysis pipeline and stored in the database. The fix engine closes the loop from diagnosis to resolution without requiring the merchant to leave the Kasparro dashboard.

Fix types supported:

- **Description:** Full product description rewrite — pushes to Shopify Admin REST API, mirrored locally, verified by read-back.
- **Title:** Optimised product title — auto-applied and verified.
- **Tags:** Restructured tag set aligned to AI taxonomy — auto-applied and verified.
- **Schema (manual):** JSON-LD structured data — generated and displayed with step-by-step instructions; cannot be safely automated via Shopify API without theme access.

- **Structure (manual):** HTML content section ordering — generated guidance for theme-level edits.

Fix workflow details:

- Edit before apply: Merchants can review and edit any AI-generated fix content before committing. Edited content replaces the original in the apply payload.
- Bulk apply: A single API call can apply multiple fixes at once — each processes sequentially with independent Shopify write + verify + score update per fix.
- Shopify verification: Every auto-applied fix reads back the updated field from Shopify after writing. Score improvement is only credited if the read-back confirms the value landed. Shopify rate-limit errors are caught and surfaced as 'Sync failed — check Shopify manually.'
- Idempotency: Applying a fix that is already marked 'applied' returns a 200 immediately without re-calling Shopify or re-scoring.
- Gap closure: Applying a fix marks the specific source gap as fixed. The store's pending/applied fix counts update immediately in the summary.

3.3 GEO Scanner — Real Multi-Platform Visibility Testing

The GEO (Generative Engine Optimisation) Scanner is the most differentiating feature in Kasparro. It runs real queries against five independent AI platforms simultaneously and measures whether the merchant's store is cited in AI-generated answers to buyer-intent questions derived from their actual product catalog.

This is not a simulation. Every scan fires actual API calls to live AI systems, with queries constructed from the store's real product types, categories, and price range. Results are persisted as visibility check records and aggregated into a citation rate trend that updates with each scan.

The five platforms in each scan:

- **Tavily Search:** Live web search API that returns source-cited results. Checks whether the store's domain appears as a citation in search results that AI overviews draw from. Measures web citation presence — the foundation of AI Overview inclusion.
- **SerpAPI (Google Organic):** Real Google organic results. Checks whether the store's pages surface in the search results that feed Google AI Overviews. A store that does not rank organically for its target queries cannot appear in Google AI citations.
- **AWS Bedrock (Nova Lite):** Brand knowledge and purchase-intent judge. Tests whether the foundation model associates the store's brand name with its claimed product categories and knows enough to include it in a recommendation answer.
- **OpenAI GPT-4o-mini:** Content quality judge. Tests whether the store's content quality and online presence would lead GPT to recommend it when answering a buyer question.

- **Internal AI (Gemini):** Consistency judge. Runs multiple query phrasings of the same buyer intent to test whether recommendations are stable or inconsistent across prompt variations.

Each platform result includes: whether the store was cited (boolean), citation URL if present, cited snippet from the generated answer, competitors mentioned in the answer, reasoning for the citation decision, and latency in milliseconds. Results are shown in the GEO Tracker page with per-platform score arcs and a ranked list of missed queries the merchant can use as a content optimisation backlog.

The overall GEO score is the citation rate across all queries and all platforms (cited results / total results × 100). This score is tracked over time — each scan batch writes visibility check records that accumulate into a history the merchant can view and export.

Why 5 platforms, not 1?

A single AI agent's citation is an anecdote. Five independent platforms — search, foundation model, content judge, brand judge, consistency judge — constitute a statistically meaningful signal about real-world AI visibility. A store that scores well on all five has genuine, robust AI discoverability, not a lucky single citation.

3.4 Competitor Intelligence Analyzer

The Competitor Intelligence feature allows a merchant to enter up to 5 competitor Shopify domain names and receive a full AI readiness analysis of each competitor's public product catalog — benchmarked directly against their own scores across all five dimensions.

For each competitor domain, the analyzer:

- Fetches the competitor's public product catalog via Shopify's storefront
- Runs the same ~40-rule deterministic scoring engine against a sample of competitor products
- Computes per-dimension average scores (Clarity, Completeness, Trust, Tags, Overall)
- Identifies the competitor's top-performing and weakest products by score

The results are displayed as a multi-bar comparison chart with all 5 dimensions side-by-side for the merchant's store and each competitor. The gap insights module highlights specific areas where the merchant is outperformed — e.g., 'Competitor X scores 22 points higher on Trust — they have JSON-LD schema markup on all products; you have it on 3% of yours.'

This feature answers a question merchants have no other way to answer: "Are my competitors' product pages better optimised for AI retrieval than mine — and specifically where?"

3.5 Content Optimisation Toolkit

The Content Optimisation Toolkit is a set of three AI-powered content generation features that give merchants ready-to-use assets for improving AI discoverability. All three are accessible from the Tools page and benefit from a 24-hour cache with catalog-hash invalidation — they regenerate automatically when the product catalog changes.

LLMs.txt Generator

LLMs.txt is a machine-readable file format emerging as a standard for helping AI crawlers understand a store's identity, product range, policies, and positioning. Analogous to robots.txt for crawlers or sitemap.xml for search engines, llms.txt gives AI agents a structured, authoritative source of truth about the store.

Kasparro generates a customised llms.txt for each store based on: the store's name and domain, its stated desired positioning, the product catalog (titles, types, tags, prices), and the FAQ and policy information surfaced during the analysis pipeline. The generated file includes: store identity, product categories, pricing context, policy summaries, and a list of topics the store can authoritatively answer.

The merchant can: preview the generated content, copy it to clipboard, download the raw file, or deploy it directly to their Shopify store as a published page at /pages/llms with one click — Kasparro creates or updates the Shopify page via the Admin API automatically.

FAQ Schema Generator

FAQ Schema is structured data (JSON-LD) that explicitly tells AI agents what questions a store can answer and what those answers are. Stores that deploy FAQ Schema are significantly more likely to be cited in AI-generated answers to buyer questions — the AI can directly reference the structured Q&A rather than inferring answers from unstructured prose.

Kasparro generates FAQ Schema grounded in three sources: the store's actual product catalog (to ensure questions are relevant), the policy text bodies scraped during analysis (so answers about refunds, shipping, and returns are factually accurate), and the FAQ gap analysis from the perception pipeline (to surface questions that competitors are answering but the merchant is not). The output is copy-pasteable JSON-LD ready to embed in the Shopify theme's header.

Product AI Q&A Test

The AI Q&A Test evaluates a single product's ability to answer 10 real buyer questions — the kind of questions a consumer might type into ChatGPT or Google AI Overview. For each question, the test assesses whether the product's title, description, tags, vendor, and price provide enough information for an AI to confidently answer.

The output is an answerability score (0–100%) with a per-question breakdown: which questions the product content can answer, which it cannot, and what specific information is missing. This is the most granular diagnostic available — it shows exactly what to add to a single product description to make it AI-recommendable.

Results are cached per product for 24 hours. Products with low answerability scores automatically surface as candidates for description fixes in the main analysis.

3.6 Store Intelligence Features

Store Intelligence features analyse the catalog as a whole — not individual products — to identify systemic gaps in topical coverage, content architecture, and brand positioning that no single-product view reveals.

AI Query Simulation

The Query Simulation feature constructs a set of realistic buyer-intent queries from the store's actual product catalog and simulates how an AI shopping assistant would respond to each one — specifically, whether it would recommend the store's products in its answer.

How it works: Each product is submitted to Gemini with the query 'A shopper asks: [AI-generated query derived from product type and category]. Would you recommend this product? Why or why not?' The model returns a 'wouldRecommend' boolean and a detailed reasoning string. The success rate (recommended / total queries) is the store's AI recommendation rate — a leading indicator of actual citation performance.

Results are cached for 24 hours and invalidated automatically if the product catalog changes (detected via a SHA-256 hash of all product titles, types, and tags). This prevents merchants from seeing stale recommendations after running fixes.

Topical Authority Analyzer

Topical authority — the degree to which a store is recognised as an expert on a specific subject domain — is a significant factor in AI citation decisions. An AI agent is more likely to recommend a store that clearly covers a topic comprehensively than one that has scattered, disconnected products.

What it does: The analyzer groups the store's products into topic clusters based on semantic similarity of titles, descriptions, and types. For each cluster, it computes a coverage score: how thoroughly does the store's content address the full breadth of that topic? Thin clusters with low coverage scores represent content gaps — topics the store sells in but has not established authority on.

Internal Link Recommendations

AI agents that crawl a store's pages use internal link structure to understand product relationships and the store's navigational hierarchy. Poor internal linking signals a disorganised catalog — the opposite of topical authority.

The Internal Link Analyzer examines all products in the catalog and identifies pairs of semantically related products that are not currently cross-linked in their descriptions. For each pair, it generates a specific linking recommendation: the source product, the target product, and the suggested anchor text phrasing. This gives merchants a concrete action list for improving both AI crawlability and human navigation.

AI Perception Report

The Perception Report answers the question: 'If an AI agent had to describe this store to a shopper asking for recommendations, what would it say — and what would it get wrong?'

After the analysis pipeline runs, a full-store perception pass queries Gemini with the complete store context — product catalog, policy pages, FAQ content, desired positioning — and requests a narrative assessment. The output includes: the agent narrative (a paragraph describing how an AI perceives the store), perceived strengths (what the store does well from an AI perspective), ambiguities (claims the AI cannot verify), unanswered questions (buyer questions the store's content cannot answer), and FAQ health (how well the existing FAQ page covers buyer intent).

The Perception Report also surfaces a prioritised action plan — a ranked list of changes that would most improve the store's AI perception, ordered by estimated impact.

Tag Optimizer

Tags are the primary taxonomy signal AI agents use to classify and compare products. Poorly formatted tags — too few, inconsistent casing, category-ambiguous — make it significantly harder for AI to correctly categorise a product. The Tag Optimizer identifies all products with tag scores below threshold and generates a suggested replacement tag set for each, aligned to AI-parseable taxonomy patterns.

3.7 Listing Readiness Assessment

The Listing Readiness Assessment provides an A–F grade (with numeric score 0–100) based on 12 objective data-completeness checks. Unlike the AI Readiness Score — which measures content quality and AI-optimisation — Listing Readiness measures whether fundamental data fields are populated at all. A product can have a high AI score but fail Listing Readiness if it is missing a price, vendor, or image.

The 12 checks include: products with meaningful descriptions (>200 characters stripped), products with rich descriptions (>500 characters — the AI quality bar), products with at least one image, products with a price set, products with a product type, products with a vendor, products with 5 or more tags, average overall AI score, average tag score, average completeness score, average trust score, and presence of refund/shipping/privacy policy pages.

Each check has a weighted contribution to the overall grade. The result page shows: the letter grade, the numeric score, each check's pass/fail status with a specific recommendation, and the top 3 priority actions to move the grade up.

3.8 Scheduled Analysis & Email Reports

Merchants should not have to remember to run an analysis. Kasparro's scheduler allows fully automated, recurring analysis runs with configurable email reports — so a merchant can check their score improvement once a week without logging in.

Scheduling options:

- Frequency: daily, weekly (with day-of-week selection), or monthly (with day-of-month selection)
- Time: configurable UTC hour (0–23) — maps to the merchant's preferred working hours
- Timezone: stored per schedule for display purposes
- Enable/disable toggle: pause a schedule without deleting configuration
- Soft pause: suspend execution while preserving all schedule settings (resume with one click)

Email report configuration:

- Recipients: comma-separated list of email addresses (delivered via Resend)
- Send always, or send only when score changes, or send only on score regression
- Minimum delta threshold: set a minimum score change (e.g., 5 points) before an email fires — reduces noise from minor fluctuations
- Test email: trigger a test report delivery immediately from the settings page to confirm addresses and format

The email report includes: store name, current scores across all 7 dimensions (overall, clarity, completeness, trust, tags, consistency, policy), total product count, critical issues count, and a direct link to the dashboard. If a previous score snapshot exists, the email shows delta values (e.g., 'Overall: 64 → 71 (+7)').

3.9 Score History & Analytics Dashboard

Every analysis run — whether triggered manually, by the scheduler, or via the API — produces a score snapshot that is appended to the store's history. Snapshots are never overwritten; the history is append-only. This gives merchants a reliable trend line that reflects their actual improvement journey.

The Dashboard presents score history as an area chart (using Recharts) with time on the X-axis and overall score on the Y-axis. The chart is populated from the snapshot history, so improvements from applied fixes are reflected in the trend line as soon as the next analysis runs.

The full snapshot history is available as a paginated table (up to 100 snapshots per page) showing: run timestamp, trigger type (manual / scheduled / api), scores across all dimensions, product count, critical/medium/low issue counts, and whether an email was sent for that run. Each snapshot is independently addressable by ID for deep-linking.

CSV export: Merchants can download a Google Sheets-compatible CSV of up to 500 snapshots from a single endpoint. The CSV includes all numeric fields, suitable for building custom reporting dashboards or sharing with a client.

3.10 Activity Feed

The Activity Feed is a chronological log of every meaningful event in the store's history: analysis started, analysis completed (with product count and gap summary), fix applied (with fix title, type, and whether Shopify sync was successful), and Shopify sync failures. Each event includes a timestamp and structured metadata.

The feed serves two purposes: (1) audit trail — a merchant or agency can see exactly what changed and when; (2) debugging aid — if a fix was applied but the score did not improve as expected, the feed shows whether the Shopify write was verified or failed silently.

4. Core User Journey

The following describes the full end-to-end journey for a merchant using Kasparro for the first time, from account creation through to measurable improvement in AI citation rates.

1. **Sign up and connect identity (Google OAuth)**

The merchant visits the Kasparro landing page and clicks 'Get Started'. They authenticate with Google OAuth — no password to create. First-time sign-in creates their account automatically. They land on an empty dashboard prompting them to connect a Shopify store.

2. **Connect Shopify store (Shopify OAuth)**

They enter their Shopify store domain (e.g., mystore.myshopify.com). Kasparro generates a Shopify OAuth install URL and redirects them to the Shopify permission grant screen. After approval, the access token is returned, encrypted with AES-256-GCM, and stored in Postgres. Kasparro immediately fetches all products from the store via the Shopify GraphQL API and populates the products table (raw data only — no scoring yet).

3. **Set desired positioning (optional but recommended)**

Before running an analysis, the merchant can set a 'desired positioning' statement — a sentence describing how they want AI agents to categorise their store (e.g., 'premium sustainable activewear for women aged 25–45'). This is used by the Perception Report and LLMs.txt generator to ground AI assessments in the merchant's intent rather than inferring it from catalog data alone.

4. **Trigger AI Readiness Analysis**

The merchant clicks 'Run Analysis' from the dashboard. The request is rate-limited (5 analyses per hour per user) and supports idempotency keys for safe client-side retries. The analysis job is queued (max 1 concurrent per server — prevents resource contention), and the UI polls for status. Progress is visible. For a 50-product store, a typical analysis completes in 60–90 seconds. For a 500-product store, 5–8 minutes.

The analysis pipeline runs for each product: rule engine scoring (< 5ms), AI quality scoring via Gemini (200–800ms), gap detection, and fix generation. Store-level passes run after all products: consistency analysis, perception report, prioritised action plan. All results are written to Postgres.

5. **Review dashboard — understand the score**

The dashboard loads with: overall AI Readiness Score (0–100), dimension breakdown (Clarity, Completeness, Trust, Tags, Consistency), a score trend area chart (vs. previous runs), total product count, critical/medium/low gap counts, pending and applied fix counts, and a recent activity feed. A benchmark

comparison shows how the store's scores compare to aspirational targets (Clarity: 82, Completeness: 78, Trust: 75) or peer averages when peer data is available.

6. **Explore gaps and apply fixes**

The merchant navigates to the Fixes page. Fixes are sorted by estimated score improvement (highest first). Each fix card shows: the product it affects, the fix type, the original content, the AI-improved content, an explanation of why this change helps AI discoverability, and an ROI rationale. The merchant can edit the improved content inline, then click 'Apply'. Kasparro writes to Shopify, verifies the write, updates the score, and marks the gap as fixed — all in one click.

For merchants with many fixes pending, the Bulk Apply feature applies all pending fixes in one operation. Each fix is processed independently with its own Shopify write + verify cycle. A summary shows how many succeeded, how many failed (with reasons), and what the net score change was.

7. **Run GEO Scanner to measure real-world AI citation**

After applying fixes, the merchant runs a GEO Scan to measure how their improvements affect real AI citation rates. The scan fires buyer-intent queries against 5 AI platforms simultaneously. Results are shown as per-platform score arcs (0–100), a list of missed queries with specific gaps, and a list of competitors that appeared in the answers when the merchant's store did not. Each scan is stored and the citation rate trend line updates automatically.

8. **Enable scheduled analysis for ongoing monitoring**

From the Schedule settings page, the merchant enables weekly analysis (e.g., every Monday at 9:00 UTC). They add their email address to the recipients list and check 'Send only on regression' — so they only hear from Kasparro when their score drops, not after every routine run. They save and receive a test email immediately to confirm the format. From this point, their AI readiness is monitored automatically.

5. Key Product Decisions and Reasoning

Decision	We Chose	Because
Decision	We Chose	Because
Score architecture	5 dimensions, 60/40 rule + AI blend	Rules provide explainability and stability; AI catches nuance rules miss. Neither alone produces a score a merchant can both understand and trust.
Platform scope	Shopify-native OAuth, not a standalone tool	Shopify merchants represent a defined, reachable ICP. A platform-native integration removes the 'enter your data manually' onboarding friction that kills adoption for analysis tools.
Fix delivery	One-click Shopify Admin API write with read-back verify	The value collapses if the fix lives in a report the merchant must manually apply. Closing the loop — diagnose → fix → verify → score — is the entire product proposition.
GEO scan breadth	5 independent AI platforms per scan	A single agent's answer is an anecdote. Five platforms (search, LLM, brand judge, content judge, consistency judge) constitute a meaningful, multi-signal assessment of real AI visibility.
AI provider strategy	Gemini primary → OpenRouter → Groq cascade	Provider resilience. A single provider outage does not fail an analysis. Gemini's free tier is generous enough for hackathon-scale use; the cascade is ready for production volume.
Feature caching	24h TTL + SHA-256 catalog-hash invalidation	Expensive AI features (query simulation, topical authority, internal links) should not re-run on every page load. Hash invalidation ensures stale results are never served after catalog changes.
Concurrent analysis	Max 1 per server, 5/hour rate limit per user	Analysis is resource-intensive. Unbounded concurrency would degrade all users. Rate limiting at 5/hour is enough for legitimate use (daily/weekly audits) while preventing abuse.
Auth strategy	Google OAuth + Postgres-backed express-session	Merchants already have Google accounts. Postgres sessions survive Railway server restarts — critical for a persistent API. No

Decision	We Chose	Because
		passwords to manage, no password-reset flows to build.
Token security	AES-256-GCM encryption at rest for Shopify tokens	Shopify access tokens are permanent admin credentials. Storing them plaintext is a critical security gap. Throwing on startup when TOKEN_ENCRYPTION_KEY is absent prevents silent insecurity.
Scheduler	node-cron in-process + Postgres schedule state	External schedulers (BullMQ, Inngest) are more robust but add deployment complexity. node-cron is sufficient for hourly-granularity schedules; state in Postgres survives restarts.
Deployment split	Vercel (SPA) + Railway (Express API)	Vercel's serverless model cannot run a persistent analysis queue, background scheduler, or in-process locks. Railway provides a persistent Node.js container. VITE_API_URL bridges the two.

6. Scope Decisions — What We Chose Not to Build

No Vector Embeddings or Semantic Search

We considered building a semantic similarity engine to compare product descriptions against a corpus of high-performing content from well-ranked stores. We cut it because: (a) a managed vector database (Pinecone, pgvector at scale) adds a deployment dependency that complicates the stack without proportional benefit at catalog sizes of 50–500 products; (b) cosine similarity scores are not explainable — a merchant needs to understand specifically what to change, not just that their embedding is 0.73 away from the cluster centroid; (c) the rule engine + AI combination already identifies the specific textual issues that semantic search would surface indirectly.

No Real-Time Webhook Product Sync

Shopify sends product/update webhooks on every product save — including minor admin edits. Auto-triggering a full analysis pipeline on every webhook would be expensive (API costs), slow (2–5 minutes per full run), and noisy (a score change for every small edit would degrade the trend line's signal quality). We chose explicit analysis on demand with optional automated scheduling (daily/weekly/monthly). The merchant controls when they want a full audit, which also means the trend line reflects deliberate optimisation rounds, not noise from minor edits.

No White-Label or Multi-Tenant Reseller Tier

The agency use case is real and commercially attractive. Custom branding, sub-account management, and role-based permissions (agency admin vs. client viewer) would make Kasparro significantly more valuable for the secondary persona. We acknowledged this as a valuable future direction and deliberately kept the data model store-ownership-scoped (every query is filtered by userId and storeId) so the multi-tenant extension requires no schema migration — only new role and permission tables.

No BigCommerce or WooCommerce Support

Other e-commerce platforms represent real opportunity but would require a fundamentally different product ingestion layer. Shopify's OAuth and GraphQL APIs are standardised and well-documented; other platforms vary significantly. Building a multi-platform abstraction layer in a hackathon timeline would produce a shallow integration on all platforms rather than a deep, well-tested integration on one.

No Mobile App

The primary action cycle — reviewing an analysis output, comparing fix options, and applying changes — benefits from a large-screen layout with side-by-side content comparison. The web app is built with a responsive layout but mobile-native patterns (gesture navigation, optimised touch targets, offline support) were not prioritised. Research on the target persona confirms that store management happens primarily from desktops.

7. Tradeoffs We Encountered and How We Resolved Them

Tradeoff 1: Score Immediacy vs. Accuracy

Tension: Running AI scoring on every product at analysis time is slow (each Gemini call takes 200–800ms). Pre-scoring products at ingest time is fast but stale — if a merchant edits product content between ingest and analysis, the score reflects the old content.

Resolution: We score on analysis, not on ingest. Ingest only loads raw product data. The merchant explicitly triggers an analysis, which correctly sets expectations (a visible progress bar for a 60–90 second wait). This also means the score always reflects the current state of the product at the moment the merchant chose to audit it.

Tradeoff 2: Fix Quality vs. Fix Speed

Tension: Generating a high-quality AI rewrite for every gap on every product during analysis is time-consuming and expensive. Generating fixes on-demand (when the merchant clicks a gap) introduces latency at the moment of action — the worst time for a user to wait.

Resolution: Fixes are pre-generated during the analysis pipeline and stored in the fixes table. When the merchant clicks Apply, we write a cached, pre-computed result — zero additional AI latency. This also allows

the merchant to review and edit the fix content before committing, which is critical for brand voice preservation.

Tradeoff 3: In-Process Queue vs. Durable Queue

Tension: An in-process analysis queue (max 1 concurrent analysis per Node.js process) is simple to implement and eliminates the need for an external dependency. But it is not durable — if the Railway container restarts mid-analysis (deploy, crash, OOM), the running job is lost and the store is left in an 'analyzing' state with no recovery path.

Resolution: In-process queue for this build, with explicit crash recovery: on server startup, we query the jobs table for any job in 'running' state, mark it as 'failed', reset the store status, and write an activity log entry. The UI detects the failed state and shows a 'Resume analysis' prompt. For production scale, this slot would be replaced with BullMQ on Redis for durable job state and horizontal scaling.

Tradeoff 4: Shopify Sync Automation Scope

Tension: Merchants want all fixes applied automatically. But fix types like 'schema' (JSON-LD in theme code) and 'structure' (HTML section ordering in product templates) require modifying the Shopify theme — which requires theme access scopes and carries significant risk of breaking the storefront if done incorrectly.

Resolution: We drew an explicit automation boundary: description, title, and tags can be auto-applied and are verified by read-back. Schema and structure fixes are generated and displayed with copy-pasteable content and step-by-step manual instructions. The UI clearly labels each fix with its sync type ('Auto-apply available' vs 'Manual action required — automation not available for this fix type'). Honest UX beats overconfident automation.

Tradeoff 5: Caching Feature Results vs. Always-Fresh

Tension: Store Intelligence features (query simulation, topical authority, internal links) involve expensive AI calls that can take 10–30 seconds. Running them on every page load would be both slow and costly. Caching them for too long means merchants see stale results after applying fixes.

Resolution: 24-hour TTL cache with catalog hash invalidation. A SHA-256 hash of all product titles, types, and tags is stored alongside each cached result. On the next feature request, the current catalog hash is computed and compared. If the catalog has changed (products added, removed, or edited), the cache is invalidated and the feature runs fresh regardless of age. If unchanged, the cached result is returned with a 'cached: true' flag so the UI can surface 'Showing cached result from [timestamp].'